

Proje Tanımı

Kurgusal çamaşır makinesi şirketi **cx_wash**'ın müşteri destek ekibi için hazırlanmış bir bilgi asistanıdır. RAG yaklaşımını kullanır: önce soruyla ilgili belge bölümü bulunur, sonra dil modeli **yalnızca o bölüme dayanarak** Türkçe cevap verir. Proje; çalışan bir API, 10 Türkçe belge ve gösterim amaçlı bir arayüzden oluşur.

Fonksiyonel Gereksinimler

1. Belgeleri yükle ve aranabilir hale getir.
2. Türkçe sorulara belgelerden cevap ver.
3. Cevapta kullanılan belgeyi ve bölümü göster.
4. Bilgi yoksa uydurma; cevap veremediğini açıkça söyle.
5. Eski ve yeni sürüm çakışırsa güncel olanı seç.

Ek olarak sohbet hafızası eklendi: her soruda son 5 mesaj modele gönderilir.

Yaklaşım

Görev tanımı çok ayrıntılı değildi. Bu yüzden **iyi ve savunulabilir** bir çözüm seçtim; en iyisi olmayabilir. Önceliğim, gereksinim değiştiğinde **kolayca değiştirilebilen** bir yapı kurmaktır.

Akış: Yükle → Bölümlere ayır → Vektöre çevir → Ara → Güncel sürümü seç → Cevapla veya reddet

Sürüm seçimi ve reddetme kararı **kodda** verilir, dil modeline bırakılmaz. Böylece sonuç her seferinde tutarlıdır.

Teknik Seçimler

Bağımlılıklar	Özenle ve az sayıda seçildi.
pathlib (os yerine)	Dosya yolu işlemlerinde kullanım kolaylığı ve güvenlik.
LangChain	Yalnızca PDF ve metin okuma ile bölme için kullanıldı. Yapay zekâ akışı hiçbir çatı kullanılmadan yazıldı.
Vektör veritabanı	Bilgi miktarının ne kadar büyüyeceği belli değildi; anlamsal arama ve ölçek için gerekiyordu.
Qdrant	Açık kaynak ve güvenilir olduğu için seçildi.

Ana Fikir: Fabrika Deseni ve Soyutlama

Görevde birçok şey belirsizdi. Bu yüzden her dış bağımlılığı bir **arayüzün (soyutlamanın)** arkasına koydum ve hangi sağlayıcının kullanılacağını bir **fabrika** sınıfına bıraktım. Fabrika, seçimi ayar dosyasından (.env) okur.

Soyut arayüz	Sağlayıcılar	Fabrika
LLMInterface	OpenaiProvider, OpenRouterProvider	LLMProviderFactory
VectordbInterface	qdrantdb	VectordbProviderFactory

Yeni bir sağlayıcı eklemek **bir sınıf ve fabrikada bir satır** demektir; kodun geri kalanı değişmez.

Nesne Yönelimli Yapı

Uygulama, her biri tek bir işten sorumlu sınıflardan oluşan dört katmana ayrılmıştır:

routes	API uç noktaları; yalnızca isteği alır ve yanıtı döndürür.
controller	İş mantığı: bölümleme, sürüm seçimi, cevaplama ve reddetme.
models	Veritabanı tabloları ve sorguları.
stores	Dil modeli ve vektör veritabanı sağlayıcıları.

- **Güvenilir:** her sınıfın tek bir sorumluluğu vardır.
- **Sağlam:** bir katmandaki değişiklik diğerlerini bozmaz.
- **Temiz:** kod kolay okunur ve kolay açıklanır.

Nesne yönelimli yaklaşım ile fabrika deseni birlikte, belirsiz bir görevde **değişime hazır** bir uygulama sağladı.